

Swing-up and stabilization of a cart-pole system

In this part of the project, you will consider an underactuated system. A system is underactuated if the number of control inputs is less than the degrees of freedom. The cart-pole system has 2 degrees-of-freedom (translation of the cart and the rotation of the pole). However, only the cart is actuated, that is, there is a single control input, force acting on the cart. Despite having less inputs than the degrees-of-freedom, the system can still be stabilized.

Your task in this part of the project is to compute a swing-up trajectory using iLQR that moves the pole from its stable downright equilibrium to a capture zone. From the capture zone an LQR stabilizes the system at the unstable equilibrium corresponding to the upright configuration of the pole.

1 System model

Let p, v, θ , and ω denote the cart position, cart velocity, pendulum angle, and angular velocity, respectively. The angle is zero at upright and positive counterclockwise in the $x - y$ plane. The state is expressed by

$$x = [p \quad v \quad \theta \quad \omega]^T, \quad v = \dot{p}, \quad \omega = \dot{\theta},$$

and the control input $u = F$ is the force F applied to the cart.

The pendulum length is L and it is represented by a point mass at distance $\ell = L/2$ from the pivot. The system dynamics is given by,

$$\begin{bmatrix} M + m & -m\ell \cos \theta \\ -m\ell \cos \theta & I + m\ell^2 \end{bmatrix} \begin{bmatrix} \ddot{p} \\ \ddot{\theta} \end{bmatrix} = \begin{bmatrix} u - b_c v - m\ell \sin \theta \omega^2 \\ m g \ell \sin \theta - b_p \omega \end{bmatrix},$$

in which, M and m are the cart and pendulum masses, g is the gravitational acceleration, and b_c and b_p are the cart and pivot viscous damping coefficients.

2 Upright stabilization

The following continuous-time linear approximation at the stable upright equilibrium is supplied. Define the pendulum inertia about the pivot as $J_p = m\ell^2$ and let $\Delta = (M + m)J_p - m^2\ell^2$. The linearized dynamics are given by

$$\dot{x} = Ax + Bu,$$

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & -\frac{J_p b_c}{\Delta} & \frac{m^2 g \ell^2}{\Delta} & -\frac{m \ell b_p}{\Delta} \\ 0 & 0 & 0 & 1 \\ 0 & -\frac{m \ell b_c}{\Delta} & \frac{(M + m) m g \ell}{\Delta} & -\frac{(M + m) b_p}{\Delta} \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ \frac{J_p}{\Delta} \\ 0 \\ \frac{m \ell}{\Delta} \end{bmatrix}.$$

2.1 Task

- Obtain a discrete-time state space model for the specified sample time, assuming the input is held between updates.
- Compute the LQR gain for stabilizing the pole at the upright position. You need to pick and justify the selected weight matrices.
- Implement the feedback in the provided MATLAB Function block and apply the force limit. You may assume that the full state is measurable.
- Validate the stabilization on the nonlinear Simscape plant.

2.2 Experiments

Empirical capture zone: Empirically determine a capture zone. This is a set of initial states from which the LQR with saturated controls can stabilize the system. You can consider limiting the cart displacement as well. Vary the angle, angular velocity, cart position, and cart velocity in your investigation. Use these results to propose a capture criterion for switching from swing up to LQR.

3 Swing-up trajectory optimization

Swing-up is the process of moving the pendulum from its downward position to the unstable upright position. Because only the cart is actuated, the controller must move the cart to transfer energy to the pendulum. The goal is to bring the pendulum near upright with sufficiently small velocities so that a local stabilizing controller, such as LQR, can take over and maintain balance. For this goal, you can use the quadratic objective function:

$$J = \frac{1}{2}x[N]^T Q_f x[N] + \frac{1}{2} \sum_{k=0}^{N-1} (x[k]^T Q x[k] + u[k]^T R u[k]),$$

which penalizes deviations from the origin, as well as the control effort.

4 Task

- Complete `backward_lqr`, `forward_pass`, `trajectory_cost`, and `ilqr` in the template. The `discrete_step` and `linearize_trajectory` functions are supplied.
- Use backward recursion to compute the coefficients of the cost-to-go function (the quadratic coefficient is S and the linear coefficient is s). Determine the corresponding feedback gains K and the feedforward terms k for the control. **Note that in the provided code, the variable name for the feedforward is d to avoid conflicts with time indexing.** Clip applied forces in each forward pass before

propagating the nonlinear dynamics. The backward recursion for the LQ Tracking problem solved in iLQR is given by

$$\begin{aligned} K_k &= (R + B_k^T S_{k+1} B_k)^{-1} B_k^T S_{k+1} A_k \\ k_k &= - (R + B_k^T S_{k+1} B_k)^{-1} (R \bar{u}[k] - B_k^T s_{k+1}) \\ S_k &= Q_k + A_k^T S_{k+1} (A_k - B_k K_k) \\ s_k &= Q_k \bar{x}[k] + A_k^T s_{k+1} + A_k^T S_{k+1} B_k k_k, \end{aligned}$$

with boundary conditions $S_N = Q_f$ and $s_N = Q_f \bar{x}[N]$.

- Implement a line search, convergence and stopping criteria, and a final computation of feedback gains along the accepted nominal trajectory.

4.1 Experiments

Varying problem description: Does iLQR always return a feasible solution? Vary problem parameters such as maximum force, allowed cart displacement and investigate the effect on success.

Choosing Q , Q_f , and R : Experiment with different different weight matrices.

Choosing the horizon length: Vary the horizon length and investigate the effect on the solution.

Initial guess: How sensitive is iLQR to the initial guess? Compare at least two initial guesses in terms of success and the resulting cost.

Handling optimization failure: You can plot the iLQR solution or print failure conditions and use those to make informed decisions on the respective trade-offs. For example, if the trajectory is not accepted because the final angular velocity was outside of the capture zone, which parameters would you change?

4.2 Critical thinking

With this strategy, feasibility of the control trajectory is not imposed during optimization. How would you change your strategy if you wanted to explicitly consider constraints on state and control, and/or disturbances that might affect the swing-up?

5 Controller design and implementation

Implement sampled trajectory tracking in the provided MATLAB Function block using the control law

$$u[k] = \bar{u}[k] - K_k (x[k] - \bar{x}[k]),$$

in which $\bar{u}[k]$ and $\bar{x}[k]$ correspond to the control and state at time k along the optimized trajectories, and K_k is the gain at time k returned by iLQR algorithm. There is a digital clock with sample time T_s to keep track of simulation time (it reports time every T_s seconds).

Switch to upright LQR when the measured state satisfies your capture criterion. Keep the force limit active and external disturbance force zero. If the trajectory ends without capture, report the failure explicitly. State how angle wrapping is handled in tracking and capture checks.

5.1 Extra task:

The project requires showing a successful swing-up followed by stabilization. However, you can also consider a possible disturbance that kicks the pole out of the capture zone. You can also implement a (supervisory) controller that detects falling and initiates a swing-up followed by LQR.

5.2 Critical thinking

So far, the full state is assumed to be measurable. Similar to the actuated pendulum, you can also consider measuring p and θ alone and estimating the respective velocities. Can you do that? How?